Complete Python Mastery Guide

From Fundamentals to Advanced Concepts

with Clear Code Examples & Explanations

A Comprehensive Reference for Beginners and Beyond

Table of Contents

1. Setup & First Program

2. Variables & Data Types

3. Operators

4. Control Flow

5. Functions

6. Data Structures

7. String Mastery

8. File I/O

9. Error Handling

10. Object-Oriented Programming

11. Modules & Packages

12. Advanced Concepts

13. Popular Libraries

1. Setup & First Program

Python comes pre-installed on macOS and most Linux distributions. For Windows, download it from python.org. Python uses indentation to define code blocks - no curly braces or semicolons needed.

hello.py

```
# This is your first Python program
print("Hello, World!")

# Python uses indentation to define blocks
if 5 > 2:
    print("Five is greater than two")  # This indentation is REQUIRED
```

Note:

Use python --version in your terminal to check your installed version. Python 3.10+ is recommended for modern features like structural pattern matching.

Python is an interpreted language, which means you can run code line by line without compiling. This makes it excellent for prototyping and learning.

2. Variables & Data Types

Python is dynamically typed - you do not declare types explicitly, though type hints are supported. Variables are created when you first assign a value to them.

Basic Types

variables.py

```
# Numbers
age = 25              # int
price = 19.99         # float
complex_num = 3 + 4j  # complex

# Strings
name = "Alice"
multiline = """This is
a multiline
string"""

# Booleans & None
is_active = True
result = None       # Represents absence of value

# Type checking
print(type(age))    # <class 'int'>
print(isinstance(name, str))  # True

# Multiple assignment
x, y, z = 1, 2, 3
a = b = c = 0  # All point to same value
```

Type Casting

casting.py

```
s = "100"
num = int(s)        # String to int
flt = float(s)      # String to float
txt = str(42)       # Int to string

# Safe conversion
val = input("Enter a number: ")  # Always returns string
num = int(val) * 2
```

Note:

Python variable names must start with a letter or underscore, can contain letters, numbers, and underscores, and are

case-sensitive.

3. Operators

Python supports a wide range of operators for arithmetic, comparison, logical operations, and more.

operators.py

```
# Arithmetic Operators
a = 10 + 5    # Addition
b = 10 - 5    # Subtraction
c = 10 * 5    # Multiplication
d = 10 / 3    # Division (always returns float)
e = 10 // 3   # Floor division (returns int)
f = 10 % 3    # Modulo (remainder)
g = 10 ** 2   # Exponentiation

# Comparison Operators
print(5 == 5)   # Equal to
print(5 != 3)   # Not equal to
print(5 > 3)    # Greater than
print(5 <= 5)   # Less than or equal to

# Logical Operators
print(True and False)  # False
print(True or False)   # True
print(not True)        # False

# Walrus Operator (Python 3.8+)
if (n := len("hello")) > 3:
    print(f"Length is {n}")  # Length is 5

# Chained Comparison
x = 5
print(1 < x < 10)  # True (cleaner than 1 < x and x < 10)
```

Note:

The walrus operator := assigns and returns a value in one expression, useful for while loops and comprehensions.

4. Control Flow

Control flow statements allow you to dictate the order in which your code executes based on conditions and loops.

Conditional Statements

conditions.py

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

# Ternary operator (conditional expression)
status = "Pass" if score >= 60 else "Fail"

# Match-Case (Python 3.10+)
match grade:
    case "A":
        print("Excellent!")
    case "B" | "C":  # Multiple patterns
        print("Good job")
    case _:
        print("Keep trying")
```

Loops

loops.py

```
# For loop - iterate over sequences
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# Range - generates numbers
for i in range(5):        # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 10, 2):  # Start, Stop, Step
    print(i)  # 2, 4, 6, 8

# While loop
```

```
count = 0
```

```
while count < 5:
```

```
    print(count)
```

```
    count += 1
```

```
# Loop control
```

```
for num in range(10):
```

```
    if num == 3:
```

```
        continue    # Skip this iteration
```

```
    if num == 7:
```

```
        break       # Exit loop entirely
```

```
    print(num)
```

```
# Else with loops - runs only if loop was not broken
```

```
for i in range(3):
```

```
    print(i)
```

```
else:
```

```
    print("Loop completed successfully")
```

Note:

The else clause on a loop executes only if the loop completed normally (did not encounter a break statement).

5. Functions

Functions are defined with def. Python supports positional, keyword, default, and arbitrary arguments. Functions are first-class citizens and can be passed around like any other object.

functions.py

```
# Basic function
def greet(name):
    """This is a docstring - describes the function."""
    return f"Hello, {name}!"

# Multiple parameters with defaults
def create_user(name, age=18, active=True):
    return {"name": name, "age": age, "active": active}

# Keyword arguments (order does not matter)
user = create_user(age=25, name="Bob")

# *args - arbitrary positional arguments (tuple)
def sum_all(*numbers):
    return sum(numbers)

print(sum_all(1, 2, 3, 4))  # 10

# **kwargs - arbitrary keyword arguments (dict)
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Alice", city="NYC", job="Engineer")

# Unpacking arguments
data = [1, 2, 3]
print(*data)  # Equivalent to print(1, 2, 3)

# Lambda (anonymous functions)
square = lambda x: x ** 2
print(square(5))  # 25

# Higher-order function
def apply_operation(x, operation):
    return operation(x)

print(apply_operation(4, lambda x: x * 2))
```

Warning:

Default argument values are evaluated once when the function is defined, not each time it is called. Never use mutable defaults like def func(data=[]) - use def func(data=None) instead.

6. Data Structures

Python provides several built-in data structures that are powerful, flexible, and optimized for common use cases.

Lists (Mutable, Ordered)

lists.py

```
my_list = [1, 2, 3, "hello", True]

# Accessing & Slicing
print(my_list[0])       # First item
print(my_list[-1])      # Last item
print(my_list[1:4])     # [2, 3, 'hello']
print(my_list[::2])     # Every second item

# Methods
my_list.append(4)       # Add to end
my_list.insert(1, "new") # Insert at index
my_list.remove(2)       # Remove first occurrence
popped = my_list.pop()  # Remove and return last
my_list.sort()          # Sort in place
my_list.reverse()       # Reverse in place

# List comprehension (powerful!)
squares = [x**2 for x in range(10)]
evens = [x for x in range(10) if x % 2 == 0]

# Nested comprehension
matrix = [[i*j for j in range(1,4)] for i in range(1,4)]
```

Tuples (Immutable, Ordered)

tuples.py

```
point = (3, 4)  # Immutable - cannot be changed after creation

# Tuple unpacking
x, y = point

# Multiple return values from function
def get_min_max(numbers):
    return min(numbers), max(numbers)  # Returns tuple

minimum, maximum = get_min_max([3, 1, 4, 1, 5])

# Named tuples for readability
from collections import namedtuple
```

```
Person = namedtuple('Person', ['name', 'age'])
```

```
alice = Person(name="Alice", age=30)
```

```
print(alice.name)  # Cleaner than index access
```

Dictionaries (Key-Value Pairs)

dicts.py

```
user = {
    "name": "Alice",
    "age": 30,
    "city": "New York"
}

# Accessing
print(user["name"])           # KeyError if missing
print(user.get("salary", 0))  # Safe access with default

# Methods
print(user.keys())
print(user.values())
print(user.items())  # Returns tuples of (key, value)

# Iteration
for key, value in user.items():
    print(f"{key}: {value}")

# Dictionary comprehension
squares = {x: x**2 for x in range(6)}

# Merging (Python 3.9+)
defaults = {"theme": "dark", "lang": "en"}
config = {"lang": "fr"}
merged = defaults | config  # Right side wins on conflict
```

Sets (Unique, Unordered)

sets.py

```
numbers = {1, 2, 3, 3, 3}  # Duplicates removed -> {1, 2, 3}

# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)  # Union: {1, 2, 3, 4, 5, 6}
print(a & b)  # Intersection: {3, 4}
print(a - b)  # Difference: {1, 2}
print(a ^ b)  # Symmetric difference: {1, 2, 5, 6}

# Remove duplicates from list
unique = list(set([1, 2, 2, 3, 3, 3]))
```

Note:

Sets are implemented as hash tables, making membership testing (x in set) extremely fast with O(1) average time

complexity.

7. String Mastery

Strings in Python are immutable sequences of Unicode characters. Python provides powerful methods and formatting options for string manipulation.

strings.py

```
text = "  Hello, Python World!  "

# Methods
print(text.lower())       # lowercase
print(text.upper())       # UPPERCASE
print(text.strip())       # Remove whitespace
print(text.replace("Python", "Amazing"))
print(text.split(","))    # Split into list

# Checking content
print("Python" in text)   # True
print(text.startswith("Hello"))

# F-strings (formatted string literals) - Python 3.6+
name = "Alice"
age = 30
print(f"My name is {name} and I am {age} years old")

# Formatting numbers
pi = 3.14159
print(f"Pi = {pi:.2f}")        # 2 decimal places
print(f"Number: {42:05d}")     # Zero-padded: 00042

# Raw strings (ignore escape sequences)
path = r"C:\Users\Alice\file.txt"  # No need to double backslashes

# Joining strings
words = ["Python", "is", "awesome"]
sentence = " ".join(words)
```

Note:

F-strings are the modern and recommended way to format strings in Python. They are faster and more readable than .format() or % formatting.

8. File I/O

Python makes reading and writing files straightforward using context managers that ensure files are properly closed.

files.py

```
# The modern way - context manager ensures file is closed
with open("data.txt", "r", encoding="utf-8") as file:
    content = file.read()           # Read entire file
    lines = file.readlines()        # Read into list

# Modes: 'r' read, 'w' write, 'a' append, 'x' create, 'b' binary

# Writing
with open("output.txt", "w") as f:
    f.write("Hello, World!\n")
    f.writelines(["Line 1\n", "Line 2\n"])

# Reading line by line (memory efficient for large files)
with open("big_file.txt") as f:
    for line in f:  # Iterates line by line
        print(line.strip())

# Working with JSON
import json

data = {"name": "Alice", "age": 30}

# Write JSON
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)

# Read JSON
with open("data.json") as f:
    loaded = json.load(f)

# Working with CSV
import csv

with open("data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Age"])
    writer.writerow(["Alice", 30])
```

Note:

Always use the with statement when working with files. It automatically closes the file even if exceptions occur.

9. Error Handling

Python uses exceptions to handle errors gracefully. A robust program anticipates potential failures and handles them appropriately.

errors.py

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:
    print(f"Unexpected error: {e}")
else:
    print("Success! No errors.")  # Runs only if no exception
finally:
    print("This always runs")     # Cleanup code goes here

# Raising exceptions
def validate_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

# Custom exceptions
class ValidationError(Exception):
    """Raised when input validation fails"""
    pass

# Exception chaining
try:
    int("not a number")
except ValueError as e:
    raise RuntimeError("Conversion failed") from e
```

Note:

Catch specific exceptions first, then more general ones. Catching bare Exception is acceptable at the top level of your application but avoid it in library code.

10. Object-Oriented Programming

Python supports object-oriented programming with classes, inheritance, encapsulation, and polymorphism. Everything in Python is an object.

oop.py

```
class Person:
    # Class attribute (shared by all instances)
    species = "Homo sapiens"

    # Constructor
    def __init__(self, name, age):
        self.name = name          # Instance attribute
        self._age = age           # Protected (by convention)
        self.__id = 12345       # Private (name mangled)

    # Instance method
    def introduce(self):
        return f"Hi, I'm {self.name}"

    # Class method - operates on class, not instance
    @classmethod
    def create_anonymous(cls):
        return cls("Anonymous", 0)

    # Static method - no access to class or instance
    @staticmethod
    def is_adult(age):
        return age >= 18

    # Property - controlled access to attributes
    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value < 0:
            raise ValueError("Age must be positive")
        self._age = value

    # String representation
    def __str__(self):    # User-friendly
        return f"{self.name} ({self._age})"

    def __repr__(self):   # Developer-friendly
        return f"Person(name='{self.name}', age={self._age})"

# Inheritance
class Employee(Person):
    def __init__(self, name, age, salary):
```

```
        super().__init__(name, age)  # Call parent constructor
```

```
        self.salary = salary
```

```
    # Method overriding
```

```
    def introduce(self):
```

```
        base = super().introduce()
```

```
        return f"{base} and I earn ${self.salary}"
```

```
# Usage
```

```
alice = Employee("Alice", 30, 75000)
```

```
print(alice.introduce())
```

```
alice.age = 31  # Uses property setter
```

Note:

OOP Principles in Python: Encapsulation (use _single and __double underscores), Inheritance (class Child(Parent)), Polymorphism (duck typing), and Abstraction (abc module).

11. Modules & Packages

Modules help you organize code into reusable files. Packages are directories of modules.

modules.py

```
# Import styles
import math
print(math.sqrt(16))

from math import sqrt, pi
print(sqrt(16))

import math as m        # Alias
from math import sqrt as square_root

# Creating your own module
# Save as mymodule.py, then import mymodule

# Check if running as script or imported
if __name__ == "__main__":
    print("Running directly")
else:
    print("Imported as module")

# Package structure
# mypackage/
#   __init__.py
#   module1.py
#   module2.py

# Installing external packages
# pip install requests numpy pandas
# pip freeze > requirements.txt
```

Note:

The if __name__ == "__main__": guard ensures that code only runs when the file is executed directly, not when imported as a module.

12. Advanced Concepts

These concepts separate intermediate Python programmers from advanced ones. Mastering them will make your code more efficient, readable, and Pythonic.

Iterators & Generators

generators.py

```
# Generator function - lazy evaluation, memory efficient
def count_up_to(n):
    count = 1
    while count <= n:
        yield count  # Returns value but saves state
        count += 1

# Generator expression
squares = (x**2 for x in range(1000000))  # Does not create list in memory!

# Iterator protocol
class Countdown:
    def __init__(self, start):
        self.start = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.start <= 0:
            raise StopIteration
        self.start -= 1
        return self.start + 1

# Useful built-ins with iterables
print(sum([1, 2, 3]))
print(max([1, 5, 3]))
print(any([False, True, False]))
print(all([True, True, True]))
print(enumerate(["a", "b", "c"]))  # (index, value) pairs
print(zip([1, 2], ["a", "b"]))      # [(1, 'a'), (2, 'b')]
```

Decorators

decorators.py

```
# A decorator is a function that takes a function and returns a function
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before call")
```

```
        result = func(*args, **kwargs)
```

```
    print("After call")
```

```
    return result
```

```
    return wrapper
```

```
@my_decorator
```

```
def say_hello():
```

```
    print("Hello!")
```

```
# Decorator with arguments
```

```
def repeat(times):
```

```
    def decorator(func):
```

```
        def wrapper(*args, **kwargs):
```

```
            for _ in range(times):
```

```
            result = func(*args, **kwargs)
```

```
    return result
```

```
        return wrapper
```

```
    return decorator
```

```
@repeat(3)
```

```
def greet():
```

```
    print("Hi!")
```

```
# Built-in decorators
```

```
class MyClass:
```

```
    @property
```

```
    def value(self):
```

```
    return 42
```

```
    @staticmethod
```

```
    def utility():
```

```
        pass
```

```
    @classmethod
```

```
    def factory(cls):
```

```
        return cls()
```

Context Managers

context.py

```
# Custom context manager using class
class DatabaseConnection:
    def __enter__(self):
        print("Opening connection...")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Closing connection...")

    def query(self, sql):
        print(f"Executing: {sql}")

with DatabaseConnection() as db:
    db.query("SELECT * FROM users")

# Context manager using generator (cleaner!)
from contextlib import contextmanager

@contextmanager
def managed_file(filename, mode):
    f = open(filename, mode)
    try:
        yield f
    finally:
        f.close()

with managed_file("test.txt", "w") as f:
    f.write("Hello")
```

Note:

Generators use lazy evaluation and are perfect for processing large datasets. A generator yields one item at a time instead of building the entire list in memory.

13. Popular Libraries

Python has a vast ecosystem of libraries. Here is a quick overview of the most popular ones across different domains.

libraries.py

```
# Data Science
import numpy as np      # Numerical computing
import pandas as pd     # Data manipulation

# Web Development
# pip install flask django fastapi
from flask import Flask  # Micro web framework

# Web Scraping
import requests          # HTTP requests
from bs4 import BeautifulSoup  # HTML parsing

# Automation
import os
import shutil
import subprocess

# Virtual Environments
# python -m venv myenv
# source myenv/bin/activate  (Linux/Mac)
# myenv\Scripts\activate     (Windows)

# Requirements file
# pip freeze > requirements.txt
# pip install -r requirements.txt
```

Note:

Always use virtual environments for your projects. They isolate dependencies and prevent conflicts between different projects.

Keep Learning!

Practice is everything. Try building a small project after each section - maybe a calculator, a todo app, or a web scraper. Python official documentation at docs.python.org is your best friend.